

Devolución TP1

Grupo 1

Demo:

- Está genial el permitir la navegación anónima, pero no debiéramos mandar al usuario por default al login, sino a ver el contenido, quizás mostrando todo de todas las zonas sin filtrar, y dejándolo cambiar eventualmente.
- El form de registro pierde el barrio seleccionado si hay un error en la carga del mismo.
- Validar mi correo no me deja logueado.
- Tras contactar, la congrats tiene un “Go back to forum” que redirige a <http://pawserver.it.itba.edu.ar/webapp> que es una página rota.
- El botón de “volver” en un post no lleva devuelta al foro, sino que literalmente hace un “back”, lo que podría recargar la misma página si por ejemplo acabo de publicar un comentario.
- Los posts tienen un textarea, lo que permite mensajes multilinea, pero al mostrarlo los saltos de línea son suprimidos.
 - Lo mismo aplica a los comentarios.
- Por qué mandar un mensaje no redirige al “chat” correspondiente bajo requests?
 - En esta vista, la foto de perfil no es clickeable, sólo el nombre
- Si no ofrezco servicios, por qué ir a requests muestra por default mis received requests?
- El foro muestra sólo 5 elementos por página
- Los filtros de tipo de servicio necesitan más padding entre imagen y texto
 - El orden, posición y tamaño no parece seguir un criterio, como usuario no sé donde ubicar lo que quiero salvo fuerza bruta

Código:

- No usan versiones alineadas de las dependencias. **Esto es un error conceptual grave.**

- Omiten modificadores de acceso y no parece ser por diseño.
- Los emails están en webapp, pero son referenciados en forma directa por la capa de servicios. Esto viola el modelo de capas establecido. **Esto es un error conceptual grave.**
- Las utility classes no debieran ser instanciables.
- Buen uso de validadores propios.
- Las zonas disponibles están hardcodeadas en múltiples lugares de la aplicación, lo que hace muy difícil agregar o quitar zonas.
- Los checks antes de loguear del estilo `isWarnEnabled` no están mal, pero no son necesarios si se está usando placeholders y dejando que el logger interpole.
- Los servicios no son verdaderos Facades, siendo el controller el responsable de orquestar casos de uso. Esto implica que hay lógica de negocio en el controller. **Esto es un error conceptual grave.**
- Hay métodos de implementaciones que no están anotados como `@Override`.
- Métodos como `EmailService.sendEmail` son de demasiado bajo nivel. El que lo invoca debe conocer que templates existen y qué variables acepta el mismo. Este método no debiera ser público si el servicio es un verdadero Facade.
- Hacen uso de estructuras como `Optional<List>` que sólo suman overhead. Es preferible usar simplemente `List` y en todo caso retornar una lista vacía (`Collections.emptyList()`) en lugar de nulos.
- Modelan correctamente `TokenType` como un enum, pero la aplicación internamente siempre usa `int`, perdiéndose toda garantía de integridad.

Seguimiento:

- Resta 1 punto por entregas intermedias (25-9 y 9-10)

Nota:

5

Grupo 2

Demo:

- En el sign up al poner un valor no admitido para el nombre de vecindario rompe con 500, deben hacer validaciones en forms del backend.
- Permite entrar a la pagina de perfiles de usuarios a uno no autenticado (siendo esto no deseado ya que los barrios son cerrados y privados), posiblemente haya una configuración mal en spring security.
- Muy completo para un barrio: tienen anuncios, feed, quejas, eventos en un calendario comunal, servicios ofrecidos.
- Podrían hacer sugerencias al crear tags para que no haya dos iguales solo cambiando el case.
- No funciona el botón de like sobre las quejas, parece que esta mal configurada la url a donde apunta.
- Tienen un link que no es correcto, por ejemplo, al clickear el tag de una queja se redirige al feed filtrando por ese tag en lugar de a las complains filtrando por el tag.
- Al ver los servicios ofrecidos, si hay algún user esta anotado en varios vecindarios, se muestra una vez por cada uno.
- Completo con 3 roles de usuarios: admin de vecindario, trabajador y usuario
 - Buen manejo de como agregar un usuario al barrio, requiere aceptación de admin. Si lo rechaza entonces me dejan intentar con otro barrio. Cuidado porque ese user esta autenticado.
- Estando como admin, en la vista de usuarios que solicitan entrar al barrio, deberían cambiar el botón que rechaza al usuario por algo que sea mas adecuado que "Unverified".

Código:

- Utilizan dos versiones de spring. `4.2.5.RELEASE` para la mayoría, pero `5.2.8.RELEASE` para `spring-context-support` y sus dependencias transitivas. **Esto es un error conceptual grave.**
- Loggean a STDOUT en debug en producción. **Esto es un error conceptual grave.**
- La conexión SMTP está configurada para debug en el entorno de producción. Hacer la configuración de email en la capa de servicio no le veo mucho sentido, pueden moverla junto a la demás configuración de la aplicación en webapp.

También podrían usar properties en lugar de harcodear las credenciales del mail en el código.

- Métodos como `sendHtmlMessage`, `sendSimpleMessage` y `sendMessageUsingThymeleafTemplate` en `EmailService` son de demasiado bajo nivel para ser públicos. El que lo invoca debe conocer que templates existen y qué variables acepta el mismo. Este método no debiera ser público si el servicio es un verdadero Facade
 - Recomendando: hagan un método por cada tipo de email, y incluyan el Locale del usuario como parámetro, y que los subjects sean keys de messages de internacionalización y no strings hardcodeados. Dado que UserRoles es un enum también pueden ser keys.
- Los emails pueden usar properties de internacionalización, no es necesario crear uno por cada lenguaje.
- Falta el annotation `@Async` en uno de los métodos de envío de mail:
`sendNewUserMail`
- Varios controllers tienen demasiadas responsabilidades (uno con casi 700 líneas). Debería haber una mejor separación de responsabilidades.
 - Recomendación: hagan uso de controller advice para error handling.
- Dado que el archivo schema.sql esta todo comentado, la aplicación no puede generar las tablas necesarias al ser deployado. **Esto viola lo pedido por enunciado de poder generar la aplicación completa para ejecutar.**
- Las clases utilitarias no deberían ser instanciables.
- Llamar otro DAO en el momento de mapear una row no es lo más adecuado, utilicen joins para esto.

Seguimiento:

- Las features están bien definidas.
- En los primeros sprints no crean bugs y pocos chores, es raro no tenerlos (ahora parece que lo estan usando).

Nota:

7

Grupo 3

Demo:

- En la búsqueda de torneos, las cards podrían contener información del user que organizó el torneo.
- En el detalle del torneo, si este está terminado debería mostrar podio, medallas de los jugadores, etc. También, podríamos hacer más visible en qué estado este se encuentra.
- Como host de un torneo, al ver el detalle de este, me gustaría ver los usuarios que no admití o que he “baneado”.
- Cuando un usuario visualiza el perfil de otro usuario que no ha participado en torneos, el mensaje mostrado es incorrecto. Actualmente, se muestra 'aún no participaste en ningún torneo', lo cual está enfocado hacia el usuario logueado. Falta CTA, el cual solo tiene sentido en el perfil del usuario logueado.
- Revisar como muestran las imágenes de usuarios. Hay lugares como el perfil donde se muestran ovaladas para aquellas imágenes rectangulares.
- Al editar el perfil, el username no soporta menos de 8 caracteres, y esto no es consistente con el form de crear cuenta (soporta menos). Esto ocasiona que no puedas hacer cambios en tu perfil si tenías un username con menos de 8 caracteres, me obliga a modificar el username.
- Se puede mejorar el estilo de los mails.
- Se puede hacer autologin al recuperar la contraseña con el flujo de forgot password.
- AL hacer login, el label de username/email dice permitir usar el email para hacer login pero al usarlo no funciona
- La pantalla que se muestra al host para terminar el torneo no tiene mucho sentido. Se muestra que tiene que completar una “reseña” y el podio, hay que dividirlo en dos secciones: resultados y reseña.
 - El host debería poder ser elegible para los resultados.
 - No deberían dejar subir reseñas hasta que el host suba los resultados (así los usuarios pueden validar de que los resultados son correctos, y sino usar las reseñas para comunicar el indebido uso).

- Dado que usamos una arquitectura de capas, dejemos a la capa de service coordinar la operación de más de un DAO.

Código:

- Loggean a STDOUT en debug en producción. **Esto es un error conceptual grave.**
- La conexión SMTP está configurada para debug en el entorno de producción. Hacer la configuración de email en la capa de servicio no le veo mucho sentido, pueden moverla junto a la demás configuración de la aplicación en webapp. También podrían usar properties en lugar de harcodear las credenciales del mail en el código.
- Métodos como `sendHtmlMessage`, `sendSimpleMessage` y `sendMessageWithAttachment` en `EmailService` son de demasiado bajo nivel para ser públicos. El que lo invoca debe conocer que templates existen y qué variables acepta el mismo. Este método no debiera ser público si el servicio es un verdadero Facade.
 - Recomendando: hagan un método por cada tipo de email, y incluyan el Locale del usuario como parámetro, y que los subjects sean keys de messages de internacionalización y no strings hardcodeados.
- Loguean a salida estándar sin usar un logger. **Esto es un error conceptual grave.**
- Si bien lo han usado ampliamente, falta usar el annotation `@Transactional(readOnly = true)` en algunos métodos de los servicios.
- El `ErrorController` no es un Controller, es un ControllerAdvice.
- Realizan chequeos de permisos en los controllers sin usar spring security. En muchos casos lo usan correctamente, pero parece que se olvidaron algunos. **Esto es un error conceptual grave.**
- Los servicios no son verdaderos Facades, siendo el controller el responsable de orquestar casos de uso. Esto implica que hay lógica de negocio en el controller. **Esto es un error conceptual grave.**
- Los tests no son realmente unitarios, por ej., cuando utilizan el DAO más de una vez. **Esto es un error conceptual grave.**
 - La verificación de el impacto en las tablas se puede hacer con `JdbcTestUtils.countRowsInTable`

- Hay test en los DAOs que ni siquiera prueban el DAO mismo, sino que a su utilitatio de test.
- Tienen todas las views dentro de una carpeta "helloworld".
- Hacen uso de estructuras como `Optional<List>` que sólo suman overhead. Es preferible usar simplemente `List` y en todo caso retornar una lista vacía (`Collections.emptyList()`) en lugar de nulos.

Seguimiento:

- Muestran un buen uso de la herramienta: buena separación entre los tipos: bugs, stories, chores, bien distribuidos entre los integrantes, pueden hacer uso de tags.

Nota:

5

Grupo 4

Demo:

- Validación de back pero en front no hay feedback
- Link verificación en mail falla si el mail posee un '+'
- Cuando falla el login me redirije a /login pero sin el nombre de grupo, resultando en 404 de tomcat
- Cuando doy like se actualiza el icono mostrando que di like pero los likes de la publicación no se actualizan.
- Para convertirme en proveedor de servicios tengo que ir al perfil y luego tocar "Become a provider"
- Cuando creo un servicio me pregunta hasta cuantos invitados puedo aceptar, pero esto solo es anecdótico ya que puedo seguir solicitando el servicio a pesar de que el evento posea mas invitados. Esto nunca es avisado explícitamente al proveedor de servicios lo que puede resultar en aceptar eventos con mas invitados que los que queria aceptar.
- Tira 500 el crear o editar eventos en title o description `<script> alert(22)</script>`

- Usan el idioma del browser para el i18n y a su vez almacenan el “idioma preferido” para los mails.

Código:

- Las dependencias están correctamente definidas con version en el dependency managment del pom padre y los hijos únicamente declaran las dependencias que usan sin la version.
- Las imágenes se ven bien dentro de los tags pero si accedo a la url de la imagen la intenta interpretar como html. Como ejemplo en [este](#) post se visualizan correctamente las imágenes pero [aca](#) no se ve bien.
- Hay lugares donde hacen:

```
List<SomeClass> aList = new ArrayList<>();
if (someCondition) {
    aList = somethingService.getList();
}
```

```
return aList;
```

Deberían usar empty list para evitar inicializar objetos innecesarios

- Utilizan correctamente controller advice para error handling, y transformación a errores http.
- Las constantes de paginación en los controllers deberian ir en una clase aparte.
- La lógica de paginación se debería mover a una clase utilitaria para evitar sobrecargar el controller de lógica que se repite.

```
pageCount = ( int ) Math.ceil(( double ) service.getElementCount() /
EVENT_PAGE_SIZE);

if (pageCount == 0){
    pageCount = 1;
}

if (offset > pageCount){
    offset = pageCount;
}
```

- Utilizan boxed primitives donde el null no tiene ningún sentido
- Utilizan offset para manejar la paginación, pero esto es mas propio de una DB que una WebApp, lo mas intuitivo seria utilizar page.
- Utilizan @Valid en los POST pero no en algunos GET donde también es necesario

- Bien por utilizar Excepciones propias, pero en algunos casos están haciendo try catch de `Exception`.
- Mezclan lógica en JavaScript utilizando condiciones que se resuelven del lado del server
- Buena utilización de Custom Validators
- Los mensajes de una conversación entre un proveedor y un organizador de eventos no están paginados y el sorting esta realizado en java
- La lógica de que subject, title, etc del mail se va a utilizar para un determinado estado del offering tiene mas sentido que este en el Service de Mail
- EmailService.sendMail debería ser private ya que requiere conocer de los templates existentes y poseer datos de propios del email.
- No hay testing a nivel servicios a pesar de que muchos servicios contienen lógica core del negocio. **Esto es pedido por enunciado.**
- Poseen Transactional en los DAOs y es consistente, ademas usan readOnly correctamente en los métodos de lectura
- Existen lugares donde se realiza la inyección de dependencias con Field Injection y en otros lugares con Constructor Injection y no es consistente
- Los borrados son físicos y debido al cascade podría llegar a suceder que cuando mire un evento anterior no se encuentre alguno de los servicios que contraté.
- No tiene mucho sentido que a nivel DAO llegue un `MultipartFile` en lugar de un `byte[]`

Seguimiento:

- Las features estan bien definidas
- Buen uso de bugs y chores
- Autologin no es una chore, aporta valor al usuario
- Mostrar logout como dropdown no es una chore

Nota:

Grupo 5

Demo:

- Falta internacionalización en muchos lugares de la aplicación.
- El botón para aplicar filtros de búsqueda es difícil de encontrar.
- Debería poder elegir sort by ascendente o descendente. Sólo se permite ascendente.
- Si aplica filtros la aplicación se rompe. Se muestra una pantalla blanca.
- En la card de la receta se muestran los botones de like y bookmark pero no son clickeables. Estaría bueno contar dichas funciones.
- Los comentarios se deberían mostrar primero el más reciente.
- Los urls de paginación están rotos. Por ejemplo en los comentarios, pasar de página lleva a la página de inicio.
- Las excepciones se deben mapear a páginas propias, no a las vistas default de Tomcat.
- En el home debería haber una sección de recetas recomendadas, que tengan en cuenta el usuario logueado.
- Se hace mal uso de la internacionalización. En lugar de mostrar “1 unidad”, “2 unidades”, se muestra “1 unidad/es”. Se vio en clase cómo configurarlo.
- Al armar las urls usan el toString genérico de Objeto. Por ejemplo “.../[Optional...]/...” para la url del usuario.
- Al editar una receta, si no se cambian las imágenes, son borradas y se deben cargar nuevamente.
- Sería adecuado mostrar más elementos por página.
- Sólo tienen un mail, enviando cuando se crea un usuario.
- Dicho mail no tiene CTA, ni link a la página y no está internacionalizado.
- Como el mail no tiene CTA, si pierdo la pestaña para realizar la activación de la cuenta, no puedo acceder de otra manera.
- Al activar la cuenta del usuario se produce un error 500.

- Se debería tener ciertos elementos como ingredientes y categorías en inglés y en español.
- El administrador debería poder borrar comentarios.
- Falta paginación en la vista de administrador.
- Sería más adecuado mostrar la vista de administrador como tabs y no como dos columnas.

Código:

- Utilizan dos versiones de spring. `4.3.4.RELEASE` para la mayoría, pero `4.2.5.RELEASE` para `spring-context-support` y sus dependencias transitivas. **Esto es un error conceptual grave.**
- Definen versiones de dependencias en los POMs hijos.
- Usan boxed primitives donde parece no tener sentido.
- Tienen modelos que generan texto a ser mostrado pero luego no es internacionalizado.
- Tienen texto sin internacionalizar. Además lo muestran sin utilizar `spring:message` o `c:out`.
- La aplicación es vulnerable a XSS. Por ejemplo, cuando el admin crea un ingrediente. **Esto es un error conceptual grave.**
- No utilizan una key adecuada para el remember me de spring security, siendo esta un string corto y hardcodeado. **Esto es un error conceptual grave.**
- Realizan chequeos de permisos en los controllers sin usar spring security. Además tienen funciones repetidas en lugar de llevarlas a una clase aparte. **Esto es un error conceptual grave.**
- Tienen todas las views dentro de una carpeta "helloworld".
- Ciertas excepciones podrían mapearse a un `ExceptionHandler`.
- El `ExceptionHandlerController` no es un `Controller`, es un `ControllerAdvice`.
- Realizan `get()` de Optional sin chequear antes `isPresent()`. Esto puede llevar a excepciones `NoSuchElementException`, las cuales no parecen ser catcheadas en ningún lado, ni tratadas por un `ExceptionHandler`.
- Imprimen a salida estándar sin usar un logger adecuado. Incluso imprimen stack traces completos a salida estándar. **Esto es un error conceptual grave.**

- Cuando reciben parámetros de búsqueda lo hacen sin usar un form, y además no realizan ningún tipo de validación.
- Crean excepciones con mensajes hardcodeados que luego no son internacionalizados.
- Mezclan el uso de snake_case y camelCase.
- Hacen uso de estructuras como `Optional<List>` que sólo suman overhead. Es preferible usar simplemente `List` y en todo caso retornar una lista vacía (`Collections.emptyList()`) en lugar de nulos.
- Cuando asignan una colección vacía usan `Collections.EMPTY_LIST` en lugar de `Collections.emptyList()`.
- Varios métodos de controllers define parámetros que luego no se utilizan.
- Crean excepciones custom que extienden de `Throwable`. Lo adecuado sería, dependiendo el caso, que extiendan de `Exception` o `RuntimeException`.
- Hacen uso inadecuado de `Optional`. No tiene sentido que un `getEntity(...)` reciba como parámetro `Optional<Integer>` para el número de página. En todo caso, debería asignarse un valor default.
- Tienen inconsistencias en cuanto a la inyección de dependencias. En algunos casos utilizan la inyección a nivel constructor, y en otros a nivel atributo.
- Métodos como `EmailService.sendEmail` son de demasiado bajo nivel para ser públicos. El que lo invoca debe conocer que templates existen y qué variables acepta el mismo. Este método no debiera ser público si el servicio es un verdadero Facade.
- Recorren colecciones al estilo imperativo, en lugar de usar métodos más apropiados como `for (String ingredient : ingredients)` o `ingredients.forEach();`.
- Cuando testean que se tire una excepción, testean un método que tira dos posibles excepciones, las cuales extienden de `Exception`. Pero luego en el test validan que arroje `RuntimeException`.
- En la capa de persistencia, crear o eliminar, por ejemplo un "like" utiliza el mismo método. Esto obliga a que el controller deba usar un POST para ambas acciones, y no un POST y un DELETE.
- Los tests, en su mayoría, sólo testean "happy paths". Es bastante deficiente el testing, sobre todo en la capa de persistencia.

Seguimiento:

- Las features están bien definidas.
- En ningún momento crean bugs, es raro no tenerlos.

Nota:

5

Grupo 6

Demo:

- Realizando búsquedas, parece que el botón de restablecer filtros esta mal configurado y me lleva al login.
- Se podría diferenciar los ordenamientos de puntuación externa y de los usuario de la plataforma.
- No se escapa el “%” en las búsquedas.
- Tienen búsqueda general por multiples entidades.
 - Tienen un botón de ver mas, que para las películas me lleva al buscador de películas. Sin embargo, allí no muestran que estoy filtrando por cierto termino.
- Buena cantidad de información (rating externo y interno, plataformas, cast, directores) y links en la pagina de detalle de película.
- En el detalle de una película, deberían dar un CTA cuando aun no hay reseñas.
- Sing up completo con call to action para volver a mandar el mail de verificación.
- En el perfil se puede mejorar:
 - Los call to action de cada pestaña, por ej. cuando aun no hay películas en mi watched-list o wishlist.
 - En la pestaña de watched-list, la barra de progreso no tiene sentido (porque son todas ya vistas).
- Algunos mensajes faltan internacionalizar
 - Como algunos mensajes de feedback

- Watched en el perfil, etc.
- Estando en el detalle de una película, si clickeo crear lista sobre esta misma, entonces deberían sumarlo a la lista que inicie a crear.
- Las recomendaciones de listas de películas parece que no son muy correctas.

Código:

- La conexión SMTP está configurada para debug en el entorno de producción. También podrían usar properties en lugar de harcodear las credenciales del mail en el código.
- Los mails no están correctamente implementados. Los templates existen en el módulo webapp, pero son referenciados desde la capa de servicios, lo que viola el modelo de capas. Adicionalmente, es responsabilidad de cada servicio conocer el detalle de los contenidos de cada template que desean usar, para popular correctamente todas las propiedades, por lo que no hay encapsulación alguna. **Esto es un error conceptual grave.**
 - El subject de los mails no esta internacionalizado.
- Poseen `Transactional` en los Services a nivel de método pero hay algunos casos en que falta, también faltan aquellos con `readOnly` en algunos métodos de lectura.
- Recomendación: hagan uso de controller advice para error handling.
- Tienen todas las views dentro de una carpeta "helloworld".
- La aplicación es vulnerable a XSS. Por ejemplo, si un admin crea una película. **Esto es un error conceptual grave.**
- No hay porque usar `StringBuilder` para armar muchas de las queries que hicieron, es mejor un string estatico (debido a que siempre son iguales, de la otra manera están creando nuevas instancias). Incluso hay lugares donde van armando la query y agregando argumentos al array de args, esto dificulta bastante poder seguir el código y la query.
- Realizan `get()` de Optional sin chequear antes `isPresent()`. Esto puede llevar a excepciones `NoSuchElementException`, las cuales no parecen ser catcheadas en ningún lado, ni tratadas por un exceptionHandler.
- Las relaciones N a M deberían ser con tablas intermedias, evitando la anomalía de duplicación.

- Escasos tests. **Esto es pedido por enunciado.**

Seguimiento:

- Muestran un buen y amplio uso de la herramienta: buena separación entre los tipos: bugs, stories, chores, bien distribuidos entre los integrantes, pueden hacer uso de tags.

Nota:

6

Grupo 7

Envían el correo a las 19:22, lo que constituye una entrega tardía.

Demo:

- Está raro el flujo de creación de una aplicación, ya que está partido por diseño. Primero tengo que elegir universidades y materias desde la sección de universidades o correlativas, y luego tengo que navegar a mano a applications para crear la aplicación. Las 3 secciones son siempre visibles y el usuario tiene que descubrir este orden. Falta streamlining de este proceso.
- La fecha de nacimiento no es un date, es un string formateado a mano por el usuario.
- "Argentino" -> esta nacionalidad no existe. Si son predefinidas, ¿por qué es un campo de texto libre?
- El upload de PDF me dice que todos los archivos son requeridos, pero al mismo tiempo me da un link para descargar el archivo subido
- No es posible consultar el estado de mi aplicación desde la web.
- Al admin le muestran el id de cada archivo junto al mismo, pero este dato es inútil
- Rechazar una aplicación no funciona si no seleccionan una universidad aceptada.
- Universidades no filtra por carrera con la que tienen equivalencias

- El form de aplicación no da feedback de que la misma se está guardando paso a paso / no permite guardar parcialmente; y al recargar me manda de vuelta al paso 1 incluso si toda la data está ya cargada.

Código:

- El repositorio incluye archivos autogenerados del IDE, lo que no es correcto.
- Está mal configurado el component scan de `ar.edu.itba.paw.webapp.auth`.
- No todos los error handlers setean el `@ResponseStatus` correspondiente.
- Hay una pobre división de responsabilidades entre controllers, donde básicamente un único controller hace todo menos páginas de error, y carga de archivos.
- Los controllers hacen chequeos de control de acceso, lo que debiera ser responsabilidad de Spring Security.
- Utilizan boxed primitives en escenarios donde no sólo el `null` no tiene semántica, sino que al consultarlo nunca validan contra `null`, permitiendo que surjan `NullPointerException`s.
- Los servicios no son verdaderos Facades, siendo el controller el responsable de orquestar casos de uso. Esto implica que hay lógica de negocio en el controller. **Esto es un error conceptual grave.**
- Los mails no están correctamente implementados. Los templates existen en el módulo webapp, pero son referenciados desde la capa de servicios, lo que viola el modelo de capas. Adicionalmente, es responsabilidad de cada servicio conocer el detalle de los contenidos de cada template que desean usar, para popular correctamente todas las propiedades, por lo que no hay encapsulación alguna. **Esto es un error conceptual grave.**
- Suprimen excepciones, que no son siquiera logueadas. **Esto es un error conceptual grave.**

Seguimiento:

- Resta 1 punto por entregas intermedias (25-9 y 9-10).

Nota:

5

Grupo 8

Demo:

- La pagina tarda unos 10 seg en cargar la Home siempre
- Si cambias el filtro y no das apply y cambias de pagina, la siguiente pagina utiliza el filtro que cambiaste y puede llegar a obtener una pagina sin resultados. Deberian mantener los filtros actuales para la paginación y en caso de cambiar algun filtro volver a comenzar la paginacion.
- Falta feedback en lista vacia en historial y lista de pacientes
- Si pido un turno y lo cancelo no se vuelve a rehabilitar ese turno para que lo tome otra persona
- No escapan los % en el search
- El mail de medico me lleva a appointments como paciente en lugar de medico
- Pantalla vacía de medical record
- No hay error cuando subo png en lugar de pdf
- 500 al cargar un pdf en la historia clinica
- Si tengo un turno en un dia posterior y le doy iniciar la consulta como médico igualmente me quedan en el listado hasta que pase la fecha.
- No hay sorting de los medicos en el explore
- No usan UTF-8
- Le falta I18N del title y CTA en mail
- No tienen fav icon
- Usan paginacion en path y no como query param en algunas vistas
- Si intento sacar un turno conmigo mismo me tira 409
- La pagina de schedule de un doctor esta mitad en español y mitad en ingles

Código:

- En el JSP de recuperar contraseña (el link que llega al mail) usan naked `${}` permitiendo XSS. **Esto es un error conceptual grave.**

- Utilizan mucho inline style, esto no es recomendado ya que no permite la reutilización de estilos y puede provocar inconsistencias en la página.
- Utilizan ISO en lugar de UTF
- El remember me key esta hardcodeada en código y es el string "REMEMBER ME". **Esto es un error conceptual grave.**
- Hay access based role en las paginas pero no hay control de ownership de las mismas a nivel spring security.
- Si bien utilizan ModelAttribute de logged user correctamente, es mejorable llevandolo a un controller advice en lugar de tenerlo dentro del mismo controller. Lo mismo con loggedDoctor.
Independientemente de esto no se aprovecha el model attribute y se llama al método en lugar de permitir la inyección.
- Buen uso de Custom Validators
- ExceptionHandlers repetidos en AppointmentController y ErrorController
- Hay métodos que son POST que no tienen body sino que solo se basan en el parámetro del query
- Falta @Valid en los Form dentro de los GET y en algunos POST
- Hay logica de negocio en los controllers, esto debería estar en un service. (Lógica de manejo de día de la semana, lógica de crear o no un nuevo insurance)
- La lógica de verificar que el user este verificado no se hace por spring security
- El uso de `@Qualifier` al injectar el view resolver es innecesario al tener 1 solo Bean.
- La lógica de autologin debería ser luego de verificar la cuenta y no debería estar dentro del controller
- Los servicios no son verdaderos Facades, hay lógica de negocio en los controllers. **Esto es un error conceptual grave.**
- Al realizar una acción sobre un appointment se llama al metodo de la vista de appointments en lugar de hacer un redirect, esto hace que en la url quede el path al post en lugar de la url de la lista.
- Validacion en el controller en lugar de usar @Valid
- Omiten modificadores de acceso

- Solo tienen @Transactional en los metodos que escriben, pero no utilizan @Transactional(readOnly=true) para metodos de lectura
- Se piden 2 veces las offices, una en el service y otra en el controller y se usan para lo mismo
- Hay métodos de implementaciones que no están anotados con `@Override`
- El DoctorsDao realiza inserciones a otras tablas que no son solo la tabla de doctores
- Utilizan el “a” de “18hs a 20hs” para obtener el intervalo de horas, siendo que no funcionaría en ingles (si estuviera la internacionalización porque ahora no está)
- Hay código que se utiliza en las vistas que no esta terminado (por ejemplo retorna null sin hacer nada)
- Solo hay solo 1 test de create por cada service y DAO. **El nivel de coverage es inadecuado para lo pedido por enunciado.**
- Cuando se crea un appointment se popula con campos null y string vacio en el dao
- La búsqueda de doctores filtrados es incorrecta a nivel dao, se buscan unas cosas por un lado y luego se hace el filtro por otro

Seguimiento:

- Las stories estan bien
- Buen uso de bugs, tal vez se podria estandarizar mas la forma de escribirlos
- Los filtros, cambios de frontend y paginacion no son chores
- Resta 0.5 por entrega intermedia el 9/10

Nota:

4.5

Grupo 9

Envían el correo a las 19:01, lo que constituye una entrega tardía.

Demo:

- i18n en mail no tiene el locale del user que recibe el mail sino de quien trigereo el envio.
- Y en la pagina hay lugares donde esta en ingles y otras en español
- Puedo editar publicaciones de otras personas (si cambio path)
- Usan 1 solo role
- Los estados solo son aceptar o rechazar no hay una contraoferta

Código:

- En los mensajes de i18n tienen hardcoded algunos valores en lugar de usar interpolaciones. **Esto es un error conceptual grave.**
- No hay control de ownership en spring security (como se detallo en la demo)
- Utilizan correctamente spring security para manejar usuarios no verificados
- Bien utilizados los custom validators, pero no utilizan values para parametrizar los valores, por ejemplo los content types aceptados, sino que estan hardcoded en el archivo de i18n
- Las clases utilitarias no deberían ser instanciables
- La clase AuthenticatedUserProvider podría ser simplificada con un ModelAttribute. Ademas es raro pasar un service como parámetro
- Los servicios no son verdaderos Facades. Los controllers tienen por tanto lógica de negocio que no debieran. **Esto es un error conceptual grave.**
- Se podría ser mas específico que hacer try catch de Exception
- La lógica de autologin se podría sacar del controller
- Hacen uso de estado compartido. Esto lleva a condiciones de carrera. **Esto es un error conceptual grave.**
- Se podría mejorar el create post llamando a 1 solo service que internamente separe en crear post y crear image. Es decir, implementar un verdadero Facade.
- Falta el @Valid en los GET. Si estan en los POST
- En el Controller de Offers hay lógica que convendría moverla al service para evitar sobrecargarlo.
- No hay `@Transactional` ni a nivel Service ni a nivel DAO. **Esto es un error conceptual grave.**

- La paginación de las reviews esta hecha a nivel Java y no a nivel SQL
- En lugar de devolver `Optional` de `Collection` es mejor devolver una `emptyCollection` como `emptyList()` o `emptySet()`
- El único test en service es create user, faltan tests. **Esto es pedido por enunciado.**
- Es interesante la clase QueryBuilder que armaron para simplificar el proceso de armado de queries de sql
- Los test unitarios en persistence no son unitarios. Autowirean todos los Daos en lugar de mockear los que no son de la clase bajo testeo. Lo utilizan para hacer el setUp, pero lo correcto seria hacer dichos inserts sin utilizar una clase creada por nosotros. **Esto es un error conceptual grave.**

Seguimiento:

- Las stories estan bien
- Bien por utilizar bugs y chores
- Los cambios de frontend y autologin no son chores
- Resta 0.5 por entrega intermedia el 9/10

Nota:

3*

*Se lo considera aprobado, pero tiene 1 punto de penalidades

Grupo 10

Demo:

- Cuando se realizan acciones de borrado no se pide confirmación.
- Los botones no muestran el cursor al hacer hover sobre ellos.
- En el perfil falta la opción de “descartar cambios” al editar el perfil.
- En la vista de perfil, se debería mostrar el calendario de disponibilidad horario. Se tiene mucho espacio sin aprovechar, y para llegar a dicho calendario debo hacer varios clicks.

- Se debería estandarizar el nombre de las materias. Por ejemplo “PAW” debería ser “Proyecto de Aplicaciones Web”.
- Sería adecuado popular la base de datos de producción con más datos, como materias y barrios. De esta forma mejoraría la experiencia al utilizar la aplicación.
- Se debería poder ver la lista de pedidos desde la vista “My subjects”.
- Formatear bien la hora en los mails. Se muestra “11”, debería ser “11:00”.
- Corregir el contraste de la aplicación. Sobre todo en los textos, siendo algunos de ellos muy difícil de leer.
- El filtro de “date” debería permitir seleccionar un día específico.
- Usar un único término para el estado “Pending”. Actualmente se utiliza “Pending” y “Missing” para el mismo estado, siendo confuso para el usuario”.
- Debería poder seleccionar modos de ordenamiento.
- Si hay una sola página, no mostrar los controles de paginación.
- En las vistas se debería mantener qué tab es la actual. Al realizar acciones esto se pierde y obliga al usuario a volver a dicha tab cada vez que produce un cambio.
- El profesor debería poder cancelar clases. Además debería poder enviar un mensaje con un motivo de dicha cancelación. Incluso se podría diseñar un proceso de rescheduling de la clase.
- El filtro de precio debería basar sus valores en base a los precios reales de la aplicación, y no ser un valor hardcodeado.
- No debería poder realizar pedidos de clase conmingo mismo.

Código:

- Las utility classes no deberían ser instanciables.
- Nombrar clases como `CustomX` en general es una mala práctica. Por definición, toda implementación de `X` es “custom”.
- Realizan controles de acceso por fuera de Spring Security.
- Imprimen stack traces completos a salida estándar sin usar un logger adecuado.
Esto es un error conceptual grave.

- Los servicios no son verdaderos Facades, siendo el controller el responsable de orquestar casos de uso. Esto implica que hay lógica de negocio en el controller. **Esto es un error conceptual grave.**
- El ExceptionController no es un Controller, es un ControllerAdvice.
- Mezclan snake_case y camelCase.
- Hay clases que hacen uso de servicios y usan anotaciones `@Autowired`, pero ninguno de estos funcionan por problemas de configuración. **Esto es un error conceptual grave.**
- Implementan un AuthenticationProvider que ofrece menos funcionalidad que la implementación default, trayendo además la posibilidad de bugs.
- El repositorio tiene subidos archivos generados por el IDE.
- Tienen todos los templates de mails duplicados debido a que tienen uno por idioma. Deberían hacer uso de archivos de internacionalización de la misma manera que se realiza en las views.
- Utilizan boxed primitives en escenarios donde no sólo el `null` no tiene semántica, sino que al consultarlo nunca validan contra `null`, permitiendo que surjan `NullPointerException`s.
- Métodos como `EmailService.sendEmail` son de demasiado bajo nivel para ser públicos. El que lo invoca debe conocer que templates existen y qué variables acepta el mismo. Este método no debiera ser público si el servicio es un verdadero Facade. **Esto es un error conceptual grave.**
- Implementan determinadas validaciones utilizando una lista, por ejemplo `List<String> presencial; // index0: true or false; index1:val`. Podría reducirse a usar un Boxed primitive, donde el valor null Sí tendría sentido semántico.
- Realizan fors con magic numbers.
- Para los roles de usuario utilizan Strings, en lugar de usar un Enum.
- Hacen uso incorrecto de `Optional`. Lo pasan como parámetro por todas las capas, para en algunos terminar haciendo `o.orElse(null)`.
- El testing en la capa de servicios es deficiente. Testean muy pocos métodos, siendo que existen una gran cantidad con lógica. **Esto es requerido por enunciado.**
- Utilizan `toString()` de enums en lugar de usar `name()`. `toString` se puede hacer override.

- Hacen uso de estructuras como `Optional<List>` que sólo suman overhead. Es preferible usar simplemente `List` y en todo caso retornar una lista vacía (`Collections.emptyList()`) en lugar de nulos.
- En los tests de persistencia llaman al mismo método en todos los tests para insertar ciertas entidades. Esto podría hacerse en un setup, o mejor aún usando directamente un archivo sql.

Seguimiento:

- Resta 0,5 puntos por entrega intermedia (09/10).
- Las stories están bien definidas.

Nota:

4.5

Grupo 11

Envían el correo a las 19:08, lo que constituye una entrega tardía.

Demo:

- Cuando se reinicia el token para activar la cuenta se produce un error 500. En lugar de crear un token nuevo cada vez, actualizar el actual.
- En los controles de paginación faltan links a la primera y última página.
- Sería adecuado desacoplar las materias de los cursos. De esta manera, puedo tener una misma materia, y crear cursos cada cuatrimestre/año de la misma materia; de forma similar a como funciona en la vida real.
- Se debería permitir crear el horario de la materia a través de una vista estilo calendario, y no ser simplemente un texto sin formato.
- Cuando se crea un curso, en lugar de mostrar “Step X”, mostrar un nombre del paso, por ejemplo, “información”, “integrantes”, etc.
- La vista para agregar alumnos y profesores a un curso es poco intuitiva.
- Estaría bueno mandar mails a los alumnos cuando son añadidos a un curso. Eventualmente se podría permitir activar/desactivar notificaciones por mail.
- Faltan filtros en la aplicación, por ejemplo, por fecha.

- Se deberían poder crear para un cuatrimestre/año en específico y no tener que asignar cada vez la fecha de inicio y final.
- Al subir archivos tener un shortcut para poder subirlo dentro de la carpeta, para no tener que elegir la carpeta nuevamente al crear un archivo.
- Como profesor, debería poder editar los participantes de un curso.

Código:

- Nombrar clases como `CustomX` en general es una mala práctica. Por definición, toda implementación de `X` es "custom".
- Los deletes están marcados como POST y no como DELETE.
- `ControllerHelper` es un `ControllerAdvice`, no un `Controller`.
- En algunos controllers no usan `@Autowired` para inyectar dependencias.
- Los servicios no son verdaderos Facades, siendo el controller el responsable de orquestar casos de uso. Esto implica que hay lógica de negocio en el controller. **Esto es un error conceptual grave.**
- En algunos exceptionHandlers no setean el status code correspondiente.
- La capa de servicios no tiene ningún tests, siendo que tienen lógica de negocio en las implementaciones. **Esto es requerido por enunciado.**
- Realizan `assertNotNull` de un `long`, lo cual nunca es null. Por lo que el test da siempre correcto.
- Invierten el orden de los asserts entre expected y actual. Esto genera que en caso de error, el mensaje obtenido no sea correcto y dificulta su corrección.

Seguimiento:

- Las stories están bien definidas.
- Bien por el uso de bugs.

Nota:

7

Grupo 12

Demo:

- No se debieran mostrar los controles de paginación si no hay más páginas.
- El paginado no se lleva bien con la división de carpetas y archivos de la vista en grilla. Es imposible predecir si hay o no más carpetas o archivos en las páginas siguientes.
- Marcar como favorita una carpeta desde la vista de grilla la selecciona.
- ¿Por qué puedo marcar como favoritas carpetas pero no archivos?
- En Mac puedo seleccionar múltiples archivos en rangos (con Shift-Click), pero no seleccionando archivos individualmente (con Cmd-Click)
- ¿Por qué todo tiene un UUID como identificador?
- El ranking default de ordenar por puntaje desc siempre va a mandar a las carpetas al final por no poder tener puntaje. El no agrupar carpetas y archivos en general dificulta las cosas.
- El form de subir archivo / crear carpeta usa el código de colores incorrecto para los botones de aceptar / cancelar, es inconsistente con el resto de la aplicación.
- Que el alta de nuevo contenido se haga desde perfil > materia es anti intuitivo.
- No hay forma sencilla de ver todos mis archivos o carpetas.

Código:

- Nombrar clases como `CustomX` en general es una mala práctica. Por definición, toda implementación de `X` es "custom".
- La implementación de `LoginFailureHandler` tiene condiciones de carrera. **Esto es un error conceptual grave.**
- Múltiples validators hacen uso de servicios y usan anotaciones `@Autowired`, pero ninguno de estos funcionan por problemas de configuración. **Esto es un error conceptual grave.**
- Utilizan boxed primitives en escenarios donde no sólo el `null` no tiene semántica, sino que al consultarlo nunca validan contra `null`, permitiendo que surjan `NullPointerException`s.

Seguimiento:

- Hacen un buen uso de la herramienta.

Nota:

8